Singly Linked List – Insert At A Given Position –Space Complexity and Algorithm.

***Program*: –**

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
```

```
    }
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
```

```
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
```

```
            insertAtEnd(&head, data);
            break;
      }
      case 3:
        {
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
           }

       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Space Complexity

The space complexity of the function void insertAtPosition $\begin{pmatrix} Node\ **head, int\ data, \\ int\ position \end{pmatrix}$

can be analyzed by considering both the input space and the auxiliary space used by the function.

Input Space:

The input space consists of the parameters passed to the function: $Node\ **head, int\ data,$ *and int position. These are constant space allocations, so they contribute* $O(1)$ *to the space complexity.*

Parameter-Only Definition: O(1) This definition considers only the space taken by the parameters themselves. The function receives `Node **head` (a pointer), `int data` (an integer), and `int position` (an integer). All three have a fixed, constant size. Therefore, the input space complexity is **O(1)**.

- `Node **head` (one pointer-to-pointer)
- `int data` (one scalar)
- `int position` (one scalar)

All three occupy a fixed amount of memory.

Full-Data Definition: Θ(n) This more common definition includes the entire data structure accessible from the parameters. If the linked list pointed to by `head` contains **`n` nodes** before the function call, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.In addition to the three parameters, we count the entire singly-linked list that *head may reference when the call begins. If the list already contains n nodes, the pre-existing memory in play is proportional to n.

Auxiliary Space:

The auxiliary space refers to any additional space used by the function that is not part of the input. In this function, the auxiliary space is primarily used for:

−The `Node * newNode` pointer, which is allocated using `malloc`.

This allocation is for a single node, so it contributes `O(1)` to the space complexity.

− The `Node * current` pointer, which is used to traverse the list.

This is also a single pointer, contributing `O(1)` to the space complexity.

−The integer `i`, used as a loop counter, which is a simple integer variable and contributes `O(1)` to the space complexity.

Auxiliary space is the extra memory an algorithm uses, not including the input. For this function:

1. **`Node *newNode`**: A single pointer is declared, which occupies constant space.
2. **`malloc(sizeof(Node))`**: Memory for exactly **one** new node is allocated. The size of a single `Node` is constant and does not depend on the list's length.
3. **`Node *current` and `int i`**: Inside the `else` block, a pointer and an integer are declared for traversing the list. These also take up constant space.

While the function's `while` loop iterates up to `position - 1` times (which affects its **time complexity**), it only reuses the same `current` pointer and `i` variable. No new memory is allocated inside the loop.

Since the total extra memory is for two pointers, one integer, and one allocated node, the amount is fixed. Therefore, the auxiliary space complexity is **Θ(1)**.

Auxiliary-space complexity Θ(1)

Extra working memory created by this routine itself:

• automatic pointer newNode

• automatic pointer current

• automatic integer i

• one dynamically allocated node via malloc(sizeof(Node)) - Whether you classify that node as “output” or “auxiliary,” it is a single fixed-size object.

• the while-loop only traverses; it allocates nothing.

Therefore the function’s own workspace does not grow with n.

Algorithm

$INSPOS(START, ITEM, POS)$

$\left((Node *)malloc\left(sizeof(Node)\right)\ is\ availability\ of\ space\ for\ insertion\ of\ Data,\ hence\ ,lets\ name\ it\ AVAIL.\right)$

1. $[ALLOCATE\ NEW\ NODE]\ \ Set\ NEW := AVAIL.$

2. $[MEMORY\ ALLOCATED?]\ if\ NEW = NULL, then:$

$Write: ``\ Error: Memory\ Allocation\ Failed.\ ``\ and\ EXIT\ .$

$[End\ of\ If\ structure.]$

3. $Set\ INFO[NEW] := ITEM[Copies\ new\ data\ into\ new\ node].$

4. $[USER\ INPUT\ POS(POSITION)\ GIVEN\ 1(1st\ NODE)\ ]if\ POS = 1\ , then:$

$Set\ LINK[NEW] := START\ [LINK\ of\ NEW\ node\ will\ get\ connected\ to\ ADDRESS\ of\ First\ NODE\ stored\ in\ START]$

$Set\ START := NEW\ [START\ will\ now\ point\ to\ NEW\ node]$

$Return$

$[END\ of\ If\ structure]$

5. $[\ LIST\ is\ Empty?\ ]\ If\ Head = NULL\ , then:$

$Write: ``\ Error: LIST\ is\ EMPTY.\ ``\ , Return\ and\ EXIT\ .$

$[End\ of\ If\ structure.]$

6. $[Position\ LESS\ THAN\ 1\ ?]\ if\ POS < 1\ , then:$

$Write: ``\ Error: INVALID\ Position.\ ``\ , Return\ and\ EXIT\ .$

$[When\ Position > 1\ , and\ List\ Is\ Not\ Empty]$

$ELSE$

$Set\ PTR := START\ [PTR\ is\ a\ pointer\ which\ is\ assigned\ to\ Head]$
$Set\ COUNT := 1\ [Count\ is\ an\ integer\ to\ count\ number\ of\ nodes]$

$Repeat\ while\ COUNT < (POS - 1)\ \ AND\ PTR\ \neq NULL:$
$Set\ PTR := LINK[PTR]\ [Updates\ PTR.]$
$Set\ COUNT := COUNT + 1\ [Updates\ COUNT.]$

$[END\ of\ Loop]$

7. $[\ PTR\ is\ NULL?\ ]\ if\ PTR = NULL\ , then:$

$Write: ``\ Error: INVALID\ Position.\ ``\ , Return\ and\ EXIT\ .$

$[When\ PTR\ is\ not\ NULL]$

$ELSE$

$Set\ LINK[NEW] := \ \ LINK[PTR]\ [LINK\ of\ NEW\ node\ is\ set\ to\ Updated\ LINK\ of\ PTR]$

$Set\ \ LINK[PTR] := NEW\ \left[\begin{array}{l} LINK\ of\ PTR\ is\ set\ to\ address\ of\ NEW\ node, this\ will\ move\ the\ NEW\ node \\ ,next\ to\ desired\ node\ and\ put\ the\ new\ node\ to\ the\ user\ input\ position\ . \end{array}\right]$

$[End\ of\ If\ structure.]$

$Note: 5th\ Step\ is\ optional, as\ 4rth\ Step\ put\ the\ node\ in\ first\ position, hence\ no\ need\ to\ check\ the\ Head\ or\ Start\ is\ Nullptr\ ,$

$i.e.\ List\ Is\ Empty.$